Implementation Progress Report

How each step works, in plain words and in the code
Predicting the Zero-Shot Transfer of a Promptable Video Tracker (SAM 3 on SA-FARI)

Kostiantyn Boiar • Supervisor: Dr Marwa Mahmoud
School of Computing Science, University of Glasgow

July 12, 2026

This report walks through *everything built so far*, one step at a time, and for each step it answers three questions: **what is the point** of it, **what does the code actually look like**, and **what is that code doing**, line by line, in plain words. The one-line goal of the whole project: given a new animal in a new place, **predict how well SAM 3 will track it *before* we run it**, and explain *why* — separately for **finding** the animal (`pDetA`) and **following** it (`pAssA`).

How to read each step: a **"The point"** paragraph (why it exists) → a real slice of the **code** on disk → an **"In the code"** plain walk-through of that slice → a green **Takeaway**. Diagrams and real-data charts appear where a picture explains the mechanism faster than words.

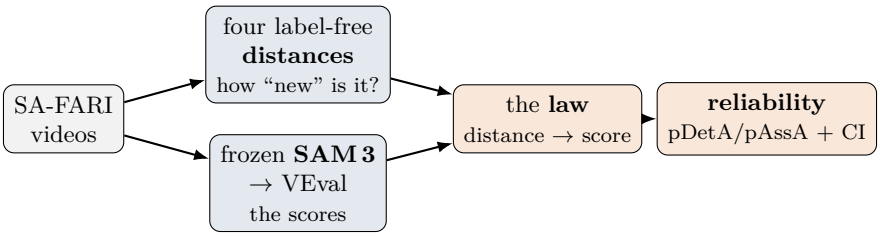

Figure 1: The whole system in one picture. The **distances** (top) are the inputs X; the frozen **SAM 3** scores (bottom) are the outputs Y; we fit a small **law** $X \to Y$ and turn it into a **reliability** estimate. The distance side is complete; the score side is built and awaiting a GPU run.

Where we are	
done	data acquisition; dataset/schema reader; the two experiments + frozen reference; all four label-free distances (taxonomic, temporal, visual, environment); the shared embedding pipeline; the leakage firewall + tests.
built	SAM 3 inference harness + VEval scoring + per-cell aggregation (run on Colab; weights access confirmed).
next	SAM 3 familiarity proxy; feature-table assembly; fit the law, cross-validation, reliability tool.

1 Step 1a — Getting the data onto disk

The point. Before we can measure anything, two things have to be on disk: the SA-FARI *annotation files* (the ground-truth boxes, masks and taxonomy) and the actual *video frames* to run the tracker on. They live in two different places with two different access rules — the annotations are gated behind a Hugging Face license click, and the frames sit in a public Google Cloud bucket. `src/acquire.py` is the single entry point that pulls both. It quietly works around a token quirk and never re-downloads anything already fetched, so a download can be stopped and restarted safely.

`src/acquire.py`

```
def _frame_uri(self, split: str, file_name: str) -> str:
    """Full bucket URI for one ``file_names`` entry under the split's frame root."""
    base = self.config.data.frames_gcs_dir.format(split=split)
```

```
    return f"{self.bucket}/{base}/{file_name}"

def fetch(self, file_names: list[str], split: str) -> int:
    # ...
    if not self.fs.exists(self._frame_uri(split, file_names[0])):
        raise FileNotFoundError(
            f"cannot locate {file_names[0]!r} at {self._frame_uri(split, file_names[0])}"
        )
    dest_root = self.config.paths.data_root / self.config.data.frames_subdir
    pulled = 0
    for name in file_names:
        dest = dest_root / name
        if dest.exists():
            continue
        dest.parent.mkdir(parents=True, exist_ok=True)
        self.fs.get_file(self._frame_uri(split, name), str(dest))
        pulled += 1
    return pulled
```

In the code. Every frame lives at a predictable web address inside Google's public bucket. `_frame_uri` builds that address: it takes a template and fills in whether we want the *train* or *test* set (`.format(split=split)`), then glues on the bucket name and the frame's relative path (something like `videoABC/frame_0007.jpg`) to produce one full location string. `fetch` then does the actual downloading, and it does three careful things. First, a **fail-fast check**: it asks the cloud filesystem "does the very first frame actually exist at this address?" — if not, it stops immediately with a clear error, because that almost always means we asked for the wrong split, and every later download would fail too. Second, the **skip-existing loop**: for each frame it computes where the file should land locally (`dest`), and if that file is already there it simply `continue`s past it — so re-running never repeats finished work, which makes the whole pull *resumable*. Third, for the frames that are genuinely missing, it creates the folder, copies the file down (`get_file`) and bumps a counter, returning how many were newly fetched so the caller reports real progress. Separately, at the top of the file, `HF_HUB_DISABLE_XET = True` is a workaround: the annotation download goes through Hugging Face, whose newer "Xet" transfer backend rejects fine-grained access tokens with a 403, so this forces the plain, reliable HTTPS path instead.

Takeaway. A resumable, split-aware downloader that lands both the gated annotations and the public frames exactly where the pipeline expects, and fails loudly on the one mistake — the wrong split — that would otherwise waste a whole run.

2 Step 1b — Reading the dataset (the schema)

The point. SA-FARI ships as one big YTVIS-style JSON per split, where species names, their 7-level taxonomy, the (video, prompt) probes, and the per-frame masks all live in *separate lists linked only by numeric ids* (Fig. 2). `src/dataset.py` turns each raw probe into one clean, typed `VideoRecord`: it resolves the species from its numeric `category_id`, flags probes where the queried animal is absent (*hard negatives*), and exposes a NaN-safe taxonomy lookup — so the rest of the pipeline works with meaningful objects instead of hunting through nested JSON. It is the single trusted place that defines what a "probe" is for the whole study.

`src/dataset.py`

```
def records(self) -> list[VideoRecord]:
    """Return every (video, prompt) probe in the split (hard negatives included by default)."""
    data = self._load()
    videos = {v["id"]: v for v in data["videos"]}
    records: list[VideoRecord] = []
    pairs = data.get("video_np_pairs")
    if pairs:
        for pair in pairs:
            video = videos.get(pair.get("video_id"))
            if video is not None:
                records.append(self._record(video, pair))
    # ... (fallback: distinct (video, category) pairs when no video_np_pairs)
```

```
    return records

def _record(self, video: dict, pair: dict) -> VideoRecord:
    """Build a VideoRecord from a video object + its video_np_pairs probe."""
    category_id = str(pair.get("category_id", ""))
    n_masklets = int(pair.get("num_masklets", 0) or 0)
    return VideoRecord(
        video_id=str(video["id"]),
        category_id=category_id,
        species=self._species_name(category_id),
        # ...
        num_masklets=n_masklets,
        is_hard_negative=(n_masklets == 0),
    )
```

In the code. `records()` walks every probe in a split. It loads the JSON once (cached) and builds a lookup dict `videos` keyed by video id, so any probe can find its video in one step instead of scanning. It then reads `video_np_pairs` — SA-FARI's official list of (video, prompt) probes, one per animal query asked of a video. For each pair it finds the matching video and hands both to `_record`; a pair that points at a missing video is simply skipped. (A fallback path, not shown, reconstructs probes from the annotations when `video_np_pairs` is absent.) `_record` does the actual conversion: it reads `category_id` — the numeric species identity — and coerces it to a string so it can serve as a stable grouping key; it reads `num_masklets`, the count of ground-truth animals of that species in that video. It then resolves two derived fields: `species` is filled by `_species_name(category_id)`, which looks the id up in the categories vocabulary; and `is_hard_negative` is `True` exactly when `num_masklets == 0` — meaning the queried species is genuinely absent, a deliberately-kept negative probe that drives the false-positive analysis, *not* a bug to filter out.

- **videos** — one entry per clip, with its frame files, camera *location*, and *when* it was filmed.
- **categories** — text concepts ("a monkey", "ocelot", …) each with a `category_id`; the real animals carry a full tree-of-life classification, most entries are generic distractors with none.
- **annotations** — the ground-truth outlines (masks), one per frame, for animals actually present.
- **probes** (`video_np_pairs`) — the questions we ask: "is *this animal* in *this video*?"; many are hard negatives.

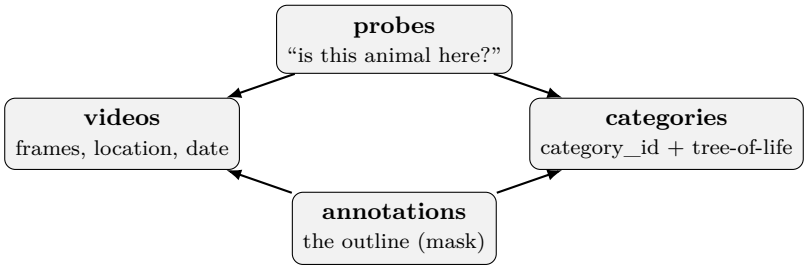

Figure 2: How SA-FARI is wired together, and what `records()` untangles. A *probe* asks whether a named animal (a `category_id`) appears in a video; *annotations* give the ground-truth outlines for the ones that do. The identity is the `category_id`, not the prompt text — two prompts can mean the same species — so everything keys on it.

Takeaway. Every raw (video, prompt) probe becomes one typed `VideoRecord` with its species resolved from `category_id` and absent-species probes flagged as hard negatives — a clean, trusted unit for the whole pipeline.

3 Step 2 — The two experiments (and the leakage firewall)

The point. The whole idea rests on a *reference* ("what we've seen") and a *probe* ("the new thing"): a distance measures how far a probe sits from the reference. Inspecting the data changed

the plan here. The official split holds out *new places* but re-uses the *same species* on both sides — so on that split "species novelty" is always zero. Because SAM 3 is **frozen** (it never trains on any of this), the split is only *our* choice of reference, and we are free to re-slice it. So we run **two complementary experiments** (Fig. 3): a **species hold-out** (leave-one-species-out — the detection/novelty test, H1) and the official **location hold-out** (the environment/association test, H2). A leakage firewall then guarantees that the axis under test is genuinely disjoint, so a later "prediction" can never be explained by the model having already seen the answer.

`src/splits.py`

```python
def build_species_partition(config: Config | None = None) -> Partition:
    """Split A - leave-one-species-out over the pooled present set."""
    cfg = config or Config()
    records = _present(pooled_records(cfg))
    species = sorted({r.category_id for r in records})
    locations = _locations(records)
    return Partition(
        name="species_holdout",
        held_axis="species",
        loso=True,
        reference_species=species,
        probe_species=species,
        reference_locations=locations,
        probe_locations=locations,
        reference_years=_years(records),
        probe_origins=["train", "test"],
    )
```

In the code. `build_species_partition` constructs Split A, the primary species hold-out. First `_present(pooled_records(cfg))` pools the train and test videos together and keeps only records that actually contain an animal (`r.num_masklets > 0`) — the positive probes. It then collects the distinct species (`category_id`) and the real location ids into sorted lists. Crucially, `reference_species` and `probe_species` are set to the *same* full list, and likewise for locations: every species is both a probe *and* part of the reference pool. The disjointness that makes this a fair test is *not* done by deleting species here — it is deferred, signalled by `loso=True` (leave-one-species-out). The idea: when we later measure how novel a given species is, we compute its distance to all the *other* reference species, excluding itself, so a species is never judged similar to itself. `held_axis="species"` records the axis under test, and `probe_origins=["train", "test"]` says probes may come from either source file (pooling ignores the official split for this experiment). The result is a small, frozen Pydantic record describing the whole experiment, saved to JSON so every downstream distance is computed against exactly this fixed anchor.

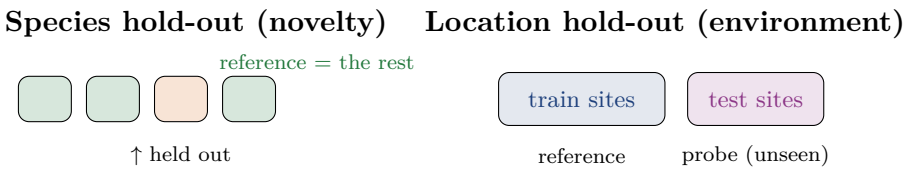

Figure 3: Two ways to slice the same data. **Left**: each species is held out in turn and compared to the others (species varies, place stays familiar). **Right**: the official geographic split (place varies, species stays familiar). Together they isolate the two hypotheses.

Takeaway. Split A defines the species-novelty experiment as one frozen reference/probe record, deferring true disjointness to a per-species leave-one-out step (`loso=True`) instead of physically removing species — and a test (Step 6) proves that firewall actually holds.

4 Step 3 — The four "how new is it?" distances

Each distance answers a plain question and is computed with *no label of the target animal*. Two of them (taxonomic, temporal) are pure lookups; two (visual, environment) share one embedding

engine.

Taxonomic — how far apart on the tree of life?

The point. Every test species needs a single number for "how unfamiliar is this animal, taxonomically, versus what the model has seen?" We answer it by measuring how far the species sits from the *nearest* training-set species on the biological tree of life, Kingdom → Species (Fig. 4). It is label-free — it uses only each species' taxonomy path, never any tracking result — so it can be computed *before* running the tracker. The self-exclusion (LOSO) rule stops a species ever counting itself as its own neighbour, which would trivially make everything look familiar (distance 0) and leak information.

`src/features/taxonomic.py`

```python
def tree_distance(a: list[str], b: list[str]) -> int:
    # same genus -> 1, same family -> 2, same order -> 3, fully disjoint -> 7
    shared = 0
    for x, y in zip(a, b, strict=False):
        if x and x == y:
            shared += 1
        else:
            break
    return len(a) - shared

# ... in TaxonomicDistance.compute:
    for cid in partition.probe_species:
        if cid not in paths:
            rows[cid] = float("nan")
            continue
        others = [paths[c] for c in reference if not (partition.loso and c == cid)]
        rows[cid] = (
            float(min(tree_distance(paths[cid], p) for p in others)) if others else float("nan")
        )
```

In the code. `tree_distance` takes two taxonomy paths, each an ordered list of names from Kingdom down to Species. It walks them level by level in lockstep: while the names match and are non-empty it counts shared ancestry (`shared += 1`); the moment they differ or a level is blank it stops. The answer is `len(a) - shared`, i.e. how many levels you must climb to reach the lowest common ancestor. So two species in the same genus differ only at the last level and score 1; sharing only the family scores 2; the order, 3; and two fully unrelated 7-level paths score 7. In `compute`, for each probe species `cid` it first checks the species has a full path (else record `NaN` and skip). It then builds `others`: the reference species' paths, but with the LOSO guard `not (partition.loso and c == cid)` dropping the probe species itself whenever the split is leave-one-species-out. Finally it takes `tree_distance` to every remaining reference species and keeps the *minimum* — the distance to the nearest known relative — falling back to `NaN` if self-exclusion emptied the reference.

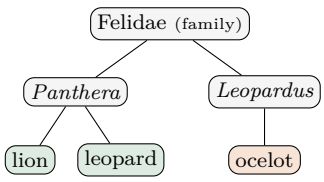

Figure 4: Lion ↔ leopard share a genus (distance 1); lion ↔ ocelot share only the family (distance 2).

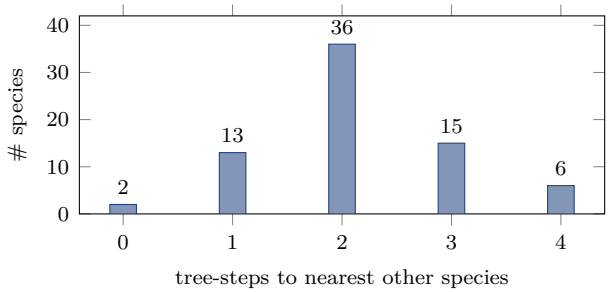

Figure 5: Measured leave-one-species-out distances (72 species). Real numbers from our run: most animals have a same-family neighbour, but a fifth are clearly novel (≥ 3). The two 0s are a quirk — "pig" and "wild boar" are the same species under two names.

Takeaway. One label-free number per test species: tree steps to its nearest training relative, with self-exclusion so a species can never be its own neighbour. Across 72 species the spread runs 0–4 (Fig. 5) — enough variation to make the novelty axis testable.

Temporal — how many years apart?

The simplest one, and a pure lookup like the taxonomic distance: the number of years between the probe's footage and the nearest reference footage (SA-FARI spans 2014 — 2024). A 2023 clip against a 2015 reference is a gap of 8. It guards against confusing "old camera hardware" with "novel animal".

Visual + environment — how unfamiliar does it *look*?

The point. The visual and environment distances work the same way: turn each animal (or scene) into a fixed-length "fingerprint" vector from a frozen vision encoder, average those into one **prototype** per species (or per site), and measure how far a probe sits from the nearest *other* prototype. Rather than duplicate this in two feature modules, the shared logic lives in one place (`pipeline.py`) so each feature only supplies the two things that actually differ: *which crop* to embed and *what to group by*. The distance is deliberately a simple, bounded "1 minus cosine similarity" so it feeds cleanly into the downstream predictor without ever peeking at a target's label.

`src/features/pipeline.py`

```python
def prototype(vectors: list[np.ndarray]) -> np.ndarray | None:
    """Re-normalised mean of L2-normalised vectors (``None`` if there are none)."""
    if not vectors:
        return None
    mean = np.mean(vectors, axis=0)
    return mean / max(float(np.linalg.norm(mean)), 1e-12)


def nearest_distance(
    vector: np.ndarray, references: dict[str, np.ndarray], exclude: str | None = None
) -> float:
    """``1 - max cosine`` to the nearest reference prototype (skipping ``exclude``); ``NaN`` if none."""
    candidates = [proto for key, proto in references.items() if key != exclude]
    if not candidates:
        return float("nan")
    return 1.0 - max(float(vector @ proto) for proto in candidates)
```

In the code. `prototype` takes a list of per-crop embedding vectors (each already unit-length from the encoder) and collapses them into a single representative direction: it averages them element-wise, then re-normalises so the result is again a unit vector pointing at that group's "average look". It returns `None` when a group produced no usable crops, which is how a `NaN` distance later surfaces honestly. `nearest_distance` then compares one probe vector against a dictionary of reference prototypes. Because all vectors are unit-length, the dot product `vector @ proto` *is* the cosine similarity (1.0 identical, 0 unrelated). It takes the *maximum* similarity over all references — the single closest neighbour — and reports $1 -$ that, so a small number means "looks familiar, close to something seen" and a larger number means "novel". The `exclude` argument is the leave-one-out guard: on the species split a probe's own prototype is also in the reference set, so it is skipped by key to stop a species matching itself and scoring 0. The `embed_crops` engine (same file) feeds these two functions: it is cache-first (only cache-miss crops trigger work), batch-fetches the missing frames per video, embeds once, writes back to the cache, and returns `{key: vector}` — the raw material `prototype` averages. The **environment** distance is the very same picture with the animal *erased* (its pixels filled with the average background) instead of cut out, grouped by location instead of species, plus a night/IR flag that fires when a frame is basically grey.

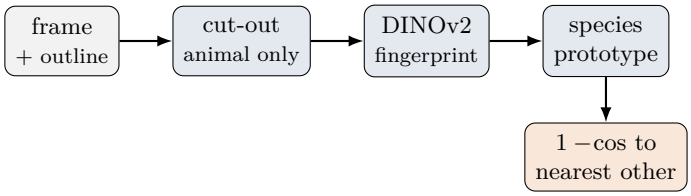

Figure 6: The visual distance, end to end — the pipeline `embed_crops` → `prototype` → `nearest_distance` draws. "Environment" is the same picture with the animal *erased* instead of cut out, grouped by location.

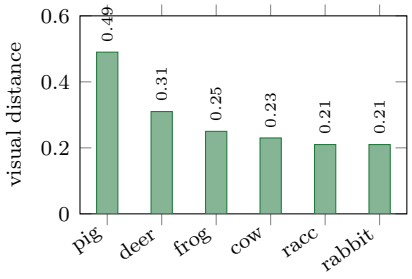

Figure 7: Measured visual distances (six-species subset). Real numbers: the pig looks most distinct, the smaller mammals cluster together.

Takeaway. One tiny, shared, label-free engine — embed once, average into prototypes, take 1 minus nearest cosine — powers both the visual and environment distances, with a leave-one-out guard so nothing matches itself. On the location hold-out the environment distances ran ≈ 0.06–0.44, with the infrared night sites flagged correctly and the daytime colour sites not.

5 Step 4a — Measuring SAM 3 (the inference harness)

The point. This is where we actually run the frozen tracker on every test clip and record what it found. **SAM 3** is a *promptable* tracker: give it a text prompt ("ocelot") and it finds and follows every matching animal across the video, with no training on that species. The step has to survive real-world interruptions: the model runs on a borrowed Colab GPU that times out mid-job, and the test split has thousands of clips. So the harness is **restartable** — it writes one small results file per video and never redoes a video it has already done. A run that dies after 400 clips picks up at clip 401 instead of starting over. Separately, the tracker is loaded only when first needed, so all the analysis code (and the tests) can import this module on a plain laptop with no GPU.

`src/inference/harness.py`

```python
def run(self, split: str = "test", limit: int | None = None) -> Path:
    """Predict masklets for every probe and write per-video JSONs; return the predictions dir."""
    out_dir = self._out_dir(split)
    out_dir.mkdir(parents=True, exist_ok=True)
    records = SAFARI(split, self.config).records()
    if limit is not None:
        records = records[:limit]

    for i, record in enumerate(records):
        path = out_dir / f"{record.video_id}.json"
        if path.exists():
            continue  # resumable: already predicted
        path.write_text(json.dumps(self._predict_video(record)))
        if (i + 1) % 50 == 0:
            print(f"  {i + 1}/{len(records)} probes predicted")

    combined = [json.loads(p.read_text()) for p in sorted(out_dir.glob("*.json"))]
    combined_path = out_dir.with_suffix(".json")
    combined_path.write_text(json.dumps({"predictions": combined}))
    print(f"predictions -> {out_dir} ({len(combined)} videos) | combined -> {combined_path}")
    return out_dir
```

In the code. Top to bottom: **(1)** `_out_dir(split)` picks an output folder keyed by *both* the split and the prompt mode (species-specific vs. the generic "animal" run), so the two experimental conditions never overwrite each other. **(2)** `SAFARI(split).records()` loads the list of every clip to run; `limit` optionally trims it to a handful for a quick smoke test. **(3)** the loop is the heart of it: for each clip it computes the output path `<video_id>.json`, and if that file already exists it does `continue` — that single `if path.exists(): continue` is what makes the whole job resumable

after a Colab timeout. **(4)** otherwise it calls `_predict_video(record)`, which loads the frames and calls `self.tracker.track(frames, prompt)` to run SAM 3, then writes the result straight to disk as JSON — so progress is saved one video at a time, not buffered in memory. **(5)** a progress line prints every 50 clips. **(6)** after the loop it re-reads all the per-video JSONs (sorted for a stable order) and concatenates them into one combined `predictions.json` that the official evaluator consumes. The per-video files are the durable source of truth; the combined file is just rebuilt from them each time, so it is always consistent with whatever has finished.

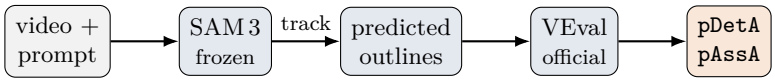

Figure 8: Measuring the ground truth. A prompt goes into frozen SAM 3, which outlines and tracks matches; the official scorer turns that into `pDetA` (*did it find the animals?*) and `pAssA` (*did it keep each identity over time?*). We analyse the two separately — that is the intellectual core of the project.

Takeaway. A crash-proof, restartable measurement run: one JSON per video plus skip-if-exists means a timed-out GPU job resumes instead of restarting, and the model only loads when actually tracking. Built and tested with a stand-in; the real run happens on a GPU.

6 Step 4b — Scoring to per-cell `pDetA` / `pAssA`

The point. The official VEval evaluator scores each individual (video, prompt) probe, but the dissertation's unit of analysis is the *cell* — a (species, location, year) group. This step is the funnel that turns many raw per-probe metrics into one clean row per cell, while also counting how much annotated data stands behind each score. That support count matters: a cell backed by 3 labelled frames should not be trusted like one backed by 300, so the later regression *weights* by it and *controls* for it — that is how "rare" is never confused with "far" (novel).

`src/eval/score.py`

```python
def aggregate(self, per_probe: dict[tuple[str, str], dict], split: str) -> pd.DataFrame:
    safari = SAFARI(split, self.config)
    support = self._support(split)
    cells: dict[tuple, dict] = defaultdict(
        lambda: {m: [] for m in _METRICS} | {"n_frames": 0, "n_masklets": 0, "n_videos": 0}
    )
    for record in safari.records():
        cell = safari.cell_of(record)
        agg = cells[(cell.category_id, cell.species, cell.location_id, cell.time)]
        metrics = per_probe.get((record.video_id, record.category_id))
        if metrics:
            for m in _METRICS:
                if metrics.get(m) is not None:
                    agg[m].append(float(metrics[m]))
        n_frames, n_masklets = support.get((record.video_id, record.category_id), (0, 0))
        agg["n_frames"] += n_frames
        agg["n_masklets"] += n_masklets
        agg["n_videos"] += 1
    # ... build one row per cell ...
    **{m: float(np.mean(agg[m])) if agg[m] else float("nan") for m in _METRICS},
```

In the code. First it asks the dataset for the support table (via `_support`) — a lookup from each probe to how many labelled frames and masklets back it. It then creates `cells`, a dictionary that auto-initialises any new cell with empty metric lists and zero counters. It walks every probe `record`; `safari.cell_of(record)` collapses that probe down to its cell key — the four coordinates (`category_id`, `species`, `location_id`, and the year from the timestamp) — so every video of the same species at the same place in the same year lands in the same bucket. It looks up that probe's VEval metrics by the `(video_id, category_id)` key; if present, each of `pDetA`/`pAssA`/`pHOTA` that is not `None` is appended to the cell's running list. Regardless of whether metrics exist, it adds this probe's frame and masklet counts to the cell's totals and bumps the video count. After the loop

each cell becomes one output row: the three metrics are collapsed to a plain mean over the probes in that cell (or `NaN` if none produced that metric), and the support counters and prompt condition are attached. The result is a tidy DataFrame with exactly one row per cell — the dependent variables the later analysis fits its "law" against.

Takeaway. Collapses many per-probe VEval scores into one row per (species, location, year) cell — the analysis unit — each carrying its own support counts so scores can be trust-weighted and "rare" is never mistaken for "far".

7 Step 5 — Fitting the law (planned)

With the distances (X) and scores (Y) in hand, the last stage fits a small, honest model:

- **A curve, not a black box.** A beta regression (a straight line through a "squashing" transform, because scores live in $[0, 1]$) maps distances to a predicted score — one model for detection, one for association, each weighted by support.
- **Which factor matters?** Variance partitioning asks whether *detection* is driven by *species novelty* and *association* by *environment* — the headline contrast.
- **Does it generalise?** We hide whole species (and whole places), predict them from distance *alone*, and check the prediction lands near the truth, with honest error bars (Fig. 9). Success is a *validated* predictor, not a higher score.
- **The tool.** Four distances in → predicted `pDetA`/`pAssA` + a confidence interval out: the practical "will it work here?" answer.

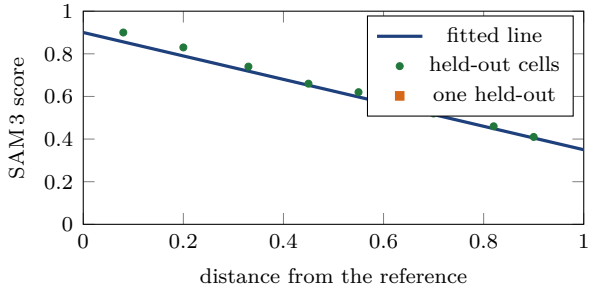

Figure 9: The target picture (*illustrative — pending the real scores*): predict a held-out cell's score from its distance and check it lands on the line, with bootstrap confidence intervals.

8 Step 6 — The rule that keeps it honest (config + tests)

The point. The whole project hinges on one rule: *no test-set information may leak into a "before you run it" prediction*, or the predictor becomes trivially perfect and the science is worthless. Two pillars enforce it. First, a single Pydantic-settings `Config` holds every constant (paths, dataset facts, inference, scoring, distances, model, cross-validation), so experiments change a *value*, never the code, and any field can be overridden by a `SAFARI_*` environment variable or a YAML file. Second, the tests turn the hard rules into checks that fail loudly. The one below proves that when we measure how "far" an unseen species is, the species is *never* compared against itself (which would fake a distance of zero).

`tests/test_leakage.py`

```python
@_needs_ann
def test_loso_excludes_self() -> None:
    """Leave-one-species-out never lets a species be its own reference (else every distance is 0)."""
    part = build_species_partition(_CFG)
    loso = TaxonomicDistance(_CFG).compute(part).dropna()
    with_self = TaxonomicDistance(_CFG).compute(part.model_copy(update={"loso": False})).dropna()

    assert (with_self == 0).all()  # self-inclusion collapses every distance to 0
    assert (loso > 0).sum() > 0    # LOSO recovers the real (mostly non-zero) distances
```

```
    assert not loso.equals(with_self)
```

In the code. This is the leave-one-species-out leakage guard, and it tests exactly the firewall from Step 2. If a species were ever allowed into its own reference set, its nearest reference would be itself, so the distance would collapse to zero — a fake, leaked answer. The test builds the species partition, then computes the distance two ways. `loso` is the correct mode: each species excluded from its own comparison set. `with_self` deliberately turns that protection off (`model_copy(update={"loso": False})`), a clean, immutable way to flip one setting on the Pydantic partition without mutating the original. The three assertions pin the behaviour down: with self-inclusion *every* distance is exactly 0 (leakage confirmed present when protection is off), with LOSO many distances are strictly positive (real structure recovered), and the two disagree. If someone ever broke the exclusion, the middle assertion would fail and the build would go red. The `_needs_ann` marker skips the test cleanly when the gated annotations are not downloaded, so the suite stays green on a fresh checkout.

Takeaway. A red-if-broken guard that the central "no test-set leakage" rule actually holds — a species can never fake a zero distance by anchoring on itself — backed by a config that makes every experiment a value change, not a code change.

9 Where we are, and the next step

The **distance side is finished and sanity-checked** on real data, and the **measurement side is built**. The codebase is typed, configuration-driven, lint-clean, and covered by a growing test suite; the label-free features were refactored and verified to reproduce their outputs exactly. The immediate milestone is the **first decision gate**: run frozen SAM 3 over the test split on Colab (weights access is confirmed), score it with VEval, aggregate to cells, and check the numbers sit in SA-FARI's published SAM 3 range. Passing that gate opens the modelling half — fitting the detection-versus-association law and building the reliability estimator.